

```
$ git init
```

Git – pierwsze kroki

Inicjowanie repozytorium · commit · historia zmian

Pracownia projektowania oprogramowania

Technik programista · klasa II · INF.04.3.5

Opracował: Bartosz Bryniarski

Czego się dzisiaj nauczysz

Cele lekcji

1

Wyjaśnisz, czym jest system kontroli wersji

2

Skonfigurujesz Gita na swoim stanowisku

3

Utworzysz repozytorium w katalogu projektu

4

Zapiszesz zmiany jako commity

5

Przejrzysz historię projektu

6

Wykluczysz zbędne pliki przez .gitignore

Znasz taki katalog?

Problem, który rozwiązuje kontrola wersji

```
projekt/  
  index.html  
  index_stary.html  
  index_KOPIA.html  
  index_final.html  
  index_final_POPRAWIONY.html  
  index_final_v2_TEN_DZIAŁA.html
```

? Który plik jest aktualny?

? Co dokładnie się zmieniło?

? Kto i kiedy to zmienił?

? Jak wrócić do wersji sprzed tygodnia?

Kopiowanie plików to nie jest system kontroli wersji.

System kontroli wersji

Definicja i korzyści

Narzędzie rejestrujące zmiany w plikach w czasie – tak, aby móc wrócić do dowolnej wcześniejszej wersji, porównać wersje i pracować równolegle w zespole.

1

Historia

Każdy zapis pracy jest odzyskiwalny

2

Odpowiedzialność

Wiadomo kto, kiedy i co zmienił

3

Zespół

Kilka osób pracuje nad jednym kodem

Skrót: VCS – Version Control System

Git w skrócie

Trzy rzeczy, które warto zapamiętać

2005

rok powstania

Autor: Linus Torvalds,
twórca jądra Linuksa

100%

historii u Ciebie

System rozproszony –
pracujesz też offline

0 zł

koszt

Wolne oprogramowanie,
standard w branży IT

Git to nie GitHub

Najczęstsza pomyłka początkujących

Git

- Program na Twoim komputerze
- Zarządza historią lokalnie
- Działa bez internetu
- Obsługa: wiersz poleceń

GitHub

- Serwis internetowy (strona WWW)
- Przechowuje repozytoria zdalnie
- Wymaga połączenia z siecią
- Alternatywy: GitLab, Bitbucket

Dzisiaj pracujemy wyłącznie lokalnie – GitHub na kolejnej lekcji.

Słownik pojęć

Bez tego ani rusz

repozytorium

Katalog projektu z ukrytym folderem .git

commit

Zapisany punkt kontrolny – migawka projektu

staging area

Poczekalnia na zmiany do najbliższego commita

hash

Unikalny identyfikator commita, np. a3f5c91

HEAD

Wskaźnik na commit, na którym jesteś

working directory

Katalog roboczy – pliki, które edytujesz

Droga pliku do historii

Serce Gita – zapamiętaj ten schemat



Analogia – paczka kurierska. Rzeczy leżą po pokoju (katalog roboczy) → wkładasz wybrane do pudełka (git add) → zaklejasz pudełko i przyklejasz karteczkę z opisem (git commit).

Konfiguracja – robisz to raz

git config

```
$ git --version
git version 2.43.0

$ git config --global user.name "Jan Kowalski"
$ git config --global user.email "jan@example.pl"

$ git config --list
```

Dlaczego?

Git podpisuje każdy commit imieniem i adresem e-mail autora. Bez tych ustawień nie zrobisz pierwszego zapisu.

--global = dla wszystkich Twoich projektów

W pracowni: na koncie współdzielonym ustaw dane BEZ --global, już wewnątrz swojego repozytorium – inaczej nadpiszesz konfigurację koledze.

git init – tworzymy repozytorium

Pierwsze polecenie w każdym nowym projekcie

```
$ mkdir wizytowka
$ cd wizytowka
$ git init
Initialized empty Git repository in
/home/uczen/wizytowka/.git/

$ ls -a
.  ..  .git
```

Powstaje katalog .git

To w nim Git trzyma całą historię projektu

Nie edytuj go ręcznie

Usunięcie .git kasuje historię bezpowrotnie

Sprawdź, gdzie jesteś

Polecenie pwd przed git init – nie zakładaj repo na pulpicie

git status – Twój najlepszy przyjaciel

Uruchamiaj po każdej operacji

```
$ git status
On branch main

Changes to be committed:
  new file:   app.py

Changes not staged for commit:
  modified:   index.html

Untracked files:
  notatki.txt
```

Untracked

Git jeszcze nie zna tego pliku

Modified

Plik zmieniony, ale nie dodany do poczekalni

Staged

Zmiana czeka w poczekalni na commit

git add i git commit

Zapisujemy punkt kontrolny

1. git add – do poczekalni

```
$ git add index.html
$ git add style.css app.js
$ git add .
# wszystkie zmiany
```

2. git commit – zapis w historii

```
$ git commit -m "Dodanie
strony glownej"

[main a3f5c91] Dodanie...
1 file changed, 9 insertions(+)
```

Bez git add nie ma commita. Git zapisuje wyłącznie to, co znalazło się w poczekalni. Przełącznik -m podaje opis commita – bez niego Git otworzy edytor tekstu.

Jak opisywać commity

Opis ma odpowiadać na pytanie: co robi ten commit?

TAK

- Dodanie formularza kontaktowego
- Poprawa walidacji adresu e-mail
- Usunięcie nieużywanych stylów CSS
- Zmiana koloru nagłówka na granatowy

NIE

- zmiany
- poprawki
- asdfgh
- ostateczna wersja final v2

Zasada: jeden commit = jedna logicznie zamknięta zmiana. Opis do ok. 50 znaków.

git log – historia projektu

Podgląd wykonanej pracy

```
$ git log --oneline
c4e1a09 Dodanie stopki z danymi kontaktowymi
9b2f7d3 Poprawa literowki w naglowku
a3f5c91 Pierwszy commit – szkielet strony

$ git show a3f5c91
commit a3f5c91... Author: Jan Kowalski
```

git log

pełna historia ze szczegółami

git log --oneline

jedna linia na commit

git log --graph

graficzna struktura gałęzi

git log -3

tylko trzy ostatnie commity

.gitignore – czego nie wrzucamy

Nie wszystko należy do repozytorium

```
# Środowisko Pythona
venv/
__pycache__/
*.pyc

# Ustawienia edytorów
.vscode/
.idea/

# Dane wrażliwe
haslo.txt
.env
```

Jedna reguła na linię

znak # rozpoczyna komentarz

Gwiazdka zastępuje tekst

*.log – wszystkie pliki .log

Ukośnik na końcu = katalog

venv/ – cały katalog

Sam .gitignore commitujemy

ma obowiązywać cały zespół

**Uwaga: .gitignore działa tylko na pliki jeszcze
nieśledzone przez Gita.**

Ściągą – polecenia z dzisiejszej lekcji

Przepisz do zeszytu

<code>git --version</code>	sprawdza wersję Gita
<code>git config --global user.name</code>	ustawia autora commitów
<code>git init</code>	tworzy nowe repozytorium
<code>git status</code>	pokazuje stan plików
<code>git add <plik></code>	dodaje plik do poczekalni
<code>git add .</code>	dodaje wszystkie zmiany
<code>git commit -m "opis"</code>	zapisuje commit
<code>git log --oneline</code>	wyświetla skróconą historię
<code>git show <hash></code>	szczegóły jednego commita

Czas na ćwiczenia

Materiał: git-lekcja-01.md · pracujemy w terminalu

- | | | | |
|---|-----------------------|--|--------|
| 1 | Konfiguracja | git --version, user.name, user.email | 5 min |
| 2 | Pierwsze repozytorium | init, status, add, commit – plik wizytówki | 10 min |
| 3 | Budowanie historii | trzy kolejne zmiany, trzy commity, git log | 10 min |
| 4 | Plik .gitignore | ukrycie notatek i katalogu tymczasowego | 7 min |
| 5 | Zadanie problemowe | analiza cudzego git status | 8 min |

Najczęstsze błędy początkujących

Sprawdź, zanim zawołasz nauczyciela



git init w złym katalogu – zawsze sprawdź pwd



Commit bez git add – Git zapisuje tylko poczekalnię



Opisy typu „zmiany” – za miesiąc nic z nich nie wynika



Jeden ogromny commit na koniec dnia



Hasła i klucze API w repozytorium – zostają w historii



Usunięcie katalogu .git – historia przepada bezpowrotnie

```
$ git log --oneline
```

Dziękuję za uwagę!

Powodzenia z pierwszym repozytorium.

Opracował: Bartosz Bryniarski